

Introduction to UML

What is UML?

Unified Modeling Language (UML)

UML is a **standard language** for specifying, visualizing, constructing, and documenting the artifacts of software systems, business modeling, and other non-software systems.

UML represents a collection of **best engineering practices** that have proven successful in the modeling of large and complex systems. It is a critical part of developing **object-oriented software** and the overall software development process.

Why use UML?

- **Communication:** Helps project teams communicate clearly.
- **Exploration:** Facilitates exploring potential designs before implementation.
- **Validation:** Allows for validating the architectural design of the software.

UML Diagrams

UML provides various diagrams to model different perspectives of a system:

Use-Case	Relation of actors to system functions.
Class	Static class structure (Attributes and Operations).
Object	Instances of classes (Snapshot of the system).
State	Life cycle and states of objects in a particular class.
Sequence	Time-ordered object message passing structure.
Collaboration	Message passing structure identifying relationships and context.
Activity	Sequential flow of activities (action states).
Component	Physical code structure and dependencies.
Component	Physical code structure and dependencies.
Deployment	Mapping of software artifacts to hardware nodes.

Scope of Class Diagrams

- **Represented:** Classes, fields, methods, and connections (interaction/inheritance).
- **Not Represented:** Algorithmic details, flow of steps, or implementation specifics.

Class Diagrams

A **Class Diagram** describes a set of objects that share the same attributes, operations, relationships, and semantics. Graphically, a class is rendered as a **rectangle** with three compartments.

Basic Class Structure

Class Representation

- **Top Compartment:** Class Name (The only required field).
- **Middle Compartment:** Attributes (State of the object).
- **Bottom Compartment:** Operations (Behavior/Methods).

Notation Tips

- **Interfaces:** Write «interface» on top of the interface name.
- **Abstract Classes:** Use *italics* for the abstract class name.

Detailed Notation

Attributes

Attributes are listed in the form: `attributeName : Type`.

- **Derived Attributes (/)** Attributes computed from others (e.g., `/age` calculated from `birthDate`).
- **Static Attributes:** Underline static attributes.
- **Multiplicity:** Defined in brackets, e.g., `name: string [1]` or `courses: Course[*]`.
- **Default Values:** e.g., `groupname: string = "staff"`.

Operations

Operations are specified by their **signature**: `name (parameter list) : returnType`

- **Static Methods:** Underline static methods.
- **Omission:** You may omit trivial (get/set) methods, but **never** omit methods from an interface.
- **Constructors:** Omit return type.

Notation Overview

Below is a visual representation of the class notation and its various components:

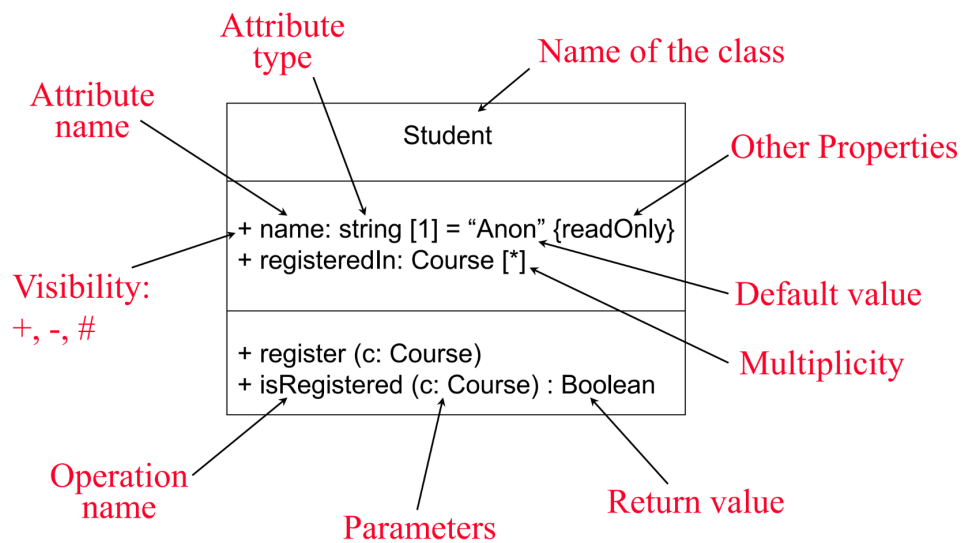


Figure 1: UML Class Notation and Component Breakdown

Visibility Modifiers

UML uses symbols to denote access levels:

Symbol	Visibility	Description
+	Public	Accessible to any class.
#	Protected	Accessible to the class and its subclasses.
-	Private	Accessible only within the class.
~	Package	Accessible within the same package.

Comments

Comments are represented as a **folded note**, attached to the appropriate class or method by a dashed line.



UML to Java Mapping

Class Example: UNIXaccount

UML Representation

UNIXaccount

```
+ username : string
+ groupname : string =
"staff"
+ filesystem_size : integer
+ creation_date : date
- password : string
- no_of_accounts : integer
= 0
```

Java Implementation

```
1 public class UNIXaccount {
2     public String username;
3     public String groupname = "csai
4         ";
5     public int filesystem_size;
6     public Date creation_date;
7     private String password;
8     static private int
9         no_of_accounts = 0;
10
11     public UNIXaccount() {
12         no_of_accounts++;
13     }
14 }
```

Operation Example: Figure

UML Representation

Figure

```
- x : integer = 0
- y : integer = 0
```

```
+ draw()
```

Java Implementation

```
1 public class Figure {
2     private int x = 0;
3     private int y = 0;
4
5     public void draw() {
6         // Java code for drawing
7     }
8 }
```

Comprehensive Example: StudentBody

StudentBody

StudentBody

```
+ main(args : String[])
```

Student

Student

```
- firstName : String
- lastName : String
- homeAddress : Address
- schoolAddress : Address
```

```
+ toString() : String
```

Tools for Creating UML Diagrams

- **Violet (Free):** Simple and easy to use.
- **Rational Rose:** Industry standard (Commercial).
- **Visual Paradigm:** Comprehensive suite (Commercial/Trial).

Usage Guidelines

Great for:

- Discovering related data and attributes.
- Getting a quick picture of important entities.
- Spotting dependencies and complexity.

Not so great for:

- Discovering algorithmic behavior.
- Finding the flow of steps (use Activity diagrams).
- Understanding overall control flow.

Summary

UML class diagrams present a **static view** of the system, displaying classes and the relationships between them. They specify the structure of how a software system will be written and function.